

3.5 for Loops

[This section corresponds to the other half of K&R Sec. 3.5]

Our second loop, which we've seen at least one example of already, is the `for` loop. The first one we saw was:

```
for (i = 0; i < 10; i = i + 1)
    printf("i is %d\n", i);
```

More generally, the syntax of a `for` loop is

```
for( expr<sub>1</sub> ; expr<sub>2</sub> ; expr<sub>3</sub> )
    statement
```

(Here we see that the `for` loop has three control expressions. As always, the *statement* can be a brace-enclosed block.)

Many loops are set up to cause some variable to step through a range of values, or, more generally, to set up an initial condition and then modify some value to perform each succeeding loop as long as some condition is true. The three expressions in a `for` loop encapsulate these conditions:

expr₁ sets up the initial condition, *expr₂* tests whether another trip through the loop should be taken, and *expr₃* increments or updates things after each trip through the loop and prior to the next one. In our first example, we had `i = 0` as *expr₁*, `i < 10` as *expr₂*, `i = i + 1` as *expr₃*, and the call to `printf` as *statement*, the body of the loop. So the loop began by setting `i` to 0, proceeded as long as `i` was less than 10, printed out `i`'s value during each trip through the loop, and added 1 to `i` between each trip through the loop.

When the compiler sees a `for` loop, first, *expr₁* is evaluated. Then, *expr₂* is evaluated, and if it is true, the body of the loop (*statement*) is executed. Then, *expr₃* is evaluated to go to the next step, and *expr₂* is evaluated again, to see if there *is* a next step. During the execution of a `for` loop, the sequence is:

```
expr<sub>1</sub>
expr<sub>2</sub>
statement
expr<sub>3</sub>
expr<sub>2</sub>
statement
expr<sub>3</sub>
...
expr<sub>2</sub>
statement
expr<sub>3</sub>
expr<sub>2</sub>
```

The first thing executed is *expr₁*. *expr₃* is evaluated after *every* trip through the loop. The last thing executed is always *expr₂*, because when *expr₂* evaluates false, the loop exits.

All three expressions of a `for` loop are optional. If you leave out *expr₁*, there simply is no initialization step, and the variable(s) used with the loop had better have been initialized already. If you leave out *expr₂*, there is no test, and the default for the `for` loop is that another trip through the loop should be taken (such that unless you break out of it some other way, the loop

runs forever). If you leave out *expr*₃, there is no increment step.

The semicolons separate the three controlling expressions of a `for` loop. (These semicolons, by the way, have nothing to do with statement terminators.) If you leave out one or more of the expressions, the semicolons remain. Therefore, one way of writing a deliberately infinite loop in C is

```
for(;;)
    ...
```

It's useful to compare C's `for` loop to the equivalent loops in other computer languages you might know. The C loop

```
for(i = x; i <= y; i = i + z)
```

is roughly equivalent to:

```
for I = X to Y step Z      (BASIC)
do 10 i=x,y,z             (FORTRAN)
for i := x to y            (Pascal)
```

In C (unlike FORTRAN), if the test condition is false before the first trip through the loop, the loop won't be traversed at all. In C (unlike Pascal), a loop control variable (in this case, `i`) is guaranteed to retain its final value after the loop completes, and it is also legal to modify the control variable within the loop, if you really want to. (When the loop terminates due to the test condition turning false, the value of the control variable after the loop will be the first value for which the condition failed, not the last value for which it succeeded.)

It's also worth noting that a `for` loop can be used in more general ways than the simple, iterative examples we've seen so far. The "control variable" of a `for` loop does not have to be an integer, and it does not have to be incremented by an additive increment. It could be "incremented" by a multiplicative factor (1, 2, 4, 8, ...) if that was what you needed, or it could be a floating-point variable, or it could be another type of variable which we haven't met yet which would step, not over numeric values, but over the elements of an array or other data structure. Strictly speaking, a `for` loop doesn't have to have a "control variable" at all; the three expressions can be anything, although the loop will make the most sense if they are related and together form the expected initialize, test, increment sequence.

The powers-of-two example of the previous section does fit this pattern, so we could rewrite it like this:

```
int x;

for(x = 2; x < 1000; x = x * 2)
    printf("%d\n", x);
```

There is no earth-shaking or fundamental difference between the `while` and `for` loops. In fact, given the general `for` loop

```
for(expr1; expr2; expr3)
    statement
```

you could usually rewrite it as a `while` loop, moving the initialize and increment expressions to statements before and within the loop:

```
expr<sub>1</sub> ;  
while(expr<sub>2</sub>)  
{  
    statement  
    expr<sub>3</sub> ;  
}
```

Similarly, given the general `while` loop

```
while(expr)  
    statement
```

you could rewrite it as a `for` loop:

```
for( ; expr; )  
    statement
```

Another contrast between the `for` and `while` loops is that although the test expression (*expr₂*) is optional in a `for` loop, it is required in a `while` loop. If you leave out the controlling expression of a `while` loop, the compiler will complain about a syntax error. (To write a deliberately infinite `while` loop, you have to supply an expression which is always nonzero. The most obvious one would simply be `while(1)` .)

If it's possible to rewrite a `for` loop as a `while` loop and vice versa, why do they both exist? Which one should you choose? In general, when you choose a `for` loop, its three expressions should all manipulate the same variable or data structure, using the initialize, test, increment pattern. If they don't manipulate the same variable or don't follow that pattern, wedging them into a `for` loop buys nothing and a `while` loop would probably be clearer. (The reason that one loop or the other can be clearer is simply that, when you see a `for` loop, you *expect* to see an idiomatic initialize/test/increment of a single variable, and if the `for` loop you're looking at doesn't end up matching that pattern, you've been momentarily misled.)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)